

From: AI Horizon Forecast aihorizonforecast@substack.com
Subject: Timer-XL: Long-Context Foundation Model for Time-Series
Forecasting
Date: 5 Jun 2025 at 23:31:09
To: egonschiele0112@gmail.com

Forwarded this email? [Subscribe here](#) for more

Timer-XL: Long-Context Foundation Model for Time-Series Forecasting

A versatile model supporting exogenous variables and varying-length context

NIKOS KAFRITSAS

JUN 5



READ IN APP 



Lately, there has been a shift in how foundation models operate.

After establishing that pretraining a large Deep Learning model on a vast

corpus of temporal data grants generalizable properties, pretrained TS models now aim to be even more versatile.

For time series, this means supporting exogenous variables and allowing variable context and prediction lengths.

This article discusses **Timer-XL[1]**, an upgraded time series model based on **Timer[2]**. Timer-XL is built for generalization, with a focus on long-context forecasting.

The model is open-source — I'll also walk through a tutorial for univariate forecasting in Part 2!

Let's get started!

✅ Find the **2 Timer-XL** notebooks in the [AI Projects folder](#) (Project 18 and Project 19)

✅ 80% of paid subscribers have stayed for over a year since I launched. If you're in it for the long run, consider switching to an **annual subscription**—you'll save 23%!

Get the Annual subscription! 📺

What is Time-XL

Time-XL is a decoder-only Transformer foundation model for forecasting. The model emphasizes generalizability and long-context predictions — offering unified, long-range forecasting.

Key features of Timer-XL:

- **Varying input/output lengths:** Unlike models such as [TTM](#) that have various versions for different input or output lengths, Time-XL uses a single model for all cases, without making assumptions about context or prediction length.
- **Long-context forecasting:** Handles longer lookback windows effectively.
- **Rich features:** Forecasts non-stationary univariate series, complex multivariate dynamics, and covariate-informed contexts with exogenous variables — all in a unified setup.
- **Versatile:** Can be trained from scratch or pretrained on large datasets. Further finetuning is optional for improved performance.

Timer-XL enhances forecasting accuracy by introducing ***TimeAttention*** — an elegant attention mechanism that we'll discuss in detail below.

The team behind Timer-XL (THUML lab at Tsinghua University[3]) has deep expertise in time-series modeling. They've released milestone models like [iTransformer](#), TimesNet, and Timer — Timer-XL's predecessor.

They also recently introduced *Sundial* — a foundation model excelling in probabilistic forecasting (we'll explore it in a future article, so stay tuned!).

Encoder vs Decoder vs Encoder-Decoder models

Before discussing Timer-XL, we'll explore the related work and recap the state of foundation TS models — this will help us understand what led to Timer-XL's breakthrough.

NLP Applications

In the early days of the Transformer, there was debate over which architecture was most effective. The original Transformer was an **Encoder-Decoder** model.

Later, the Transformer research split into 2 branches: Encoder-only, like BERT (led by Google), and decoder-only, like GPT (led by OpenAI):

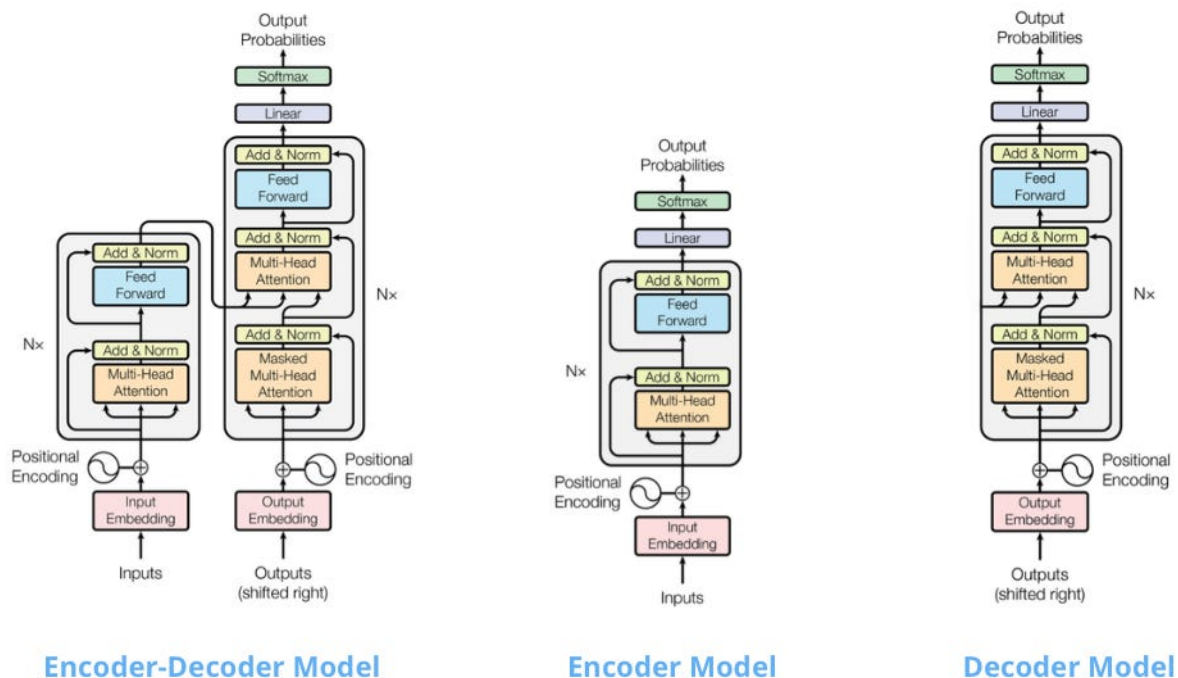


Figure 1: The initial variants of the Transformer model - **Left:** the original Transformer model from Attention is all you need, **Center:** An encoder-only model(Bert), **Right:** A decoder-only model (GPT-1) (Image annotated from [here](#))

- **Encoder-Decoder** models use a bidirectional encoder to understand input and a causal decoder to generate output one token at a time. They excel at sequence-to-sequence tasks such as translation and summarization.

- **Encoder-only models** use bidirectional attention (look for context in both ways) to understand a sentence and predict masked words within a sentence. They excel at NLU (natural language understanding) tasks.
- **Decoder-only** models use causal attention (the model only looks back for context) and learn to predict the next word. They excel at NLG (natural language generation) tasks.

In NLP, **decoder-only** models dominate generation tasks. Encoder-only models are used in classification, regression, and NER (Named Entity Recognition).

Time-Series Applications

By late 2024 and early 2025, numerous foundation models were published, providing ample evidence of what works best.

All these foundation models have come in many flavours. Examples are:

- Decoder models
 - [TimesFM](#) (Google), [Time-MOE](#)
- Encoder models
 - [MOIRAI](#) (Salesforce), [MOMENT](#)
- Encoder-Decoder
 - Chronos (Amazon)

So far, **decoder** and **encoder-decoder** models outperform **encoders** in forecasting. Timer-XL's authors back this up with extensive experiments — their results support the same conclusion.

There is also a category of multi-purpose models — used for forecasting,

classification, imputation, etc. MOMENT and UNITS belong here and are encoder-only models.

Timer, also multi-purpose, is a **decoder-only** model. Its successor, **Timer-XL**, outperforms Timer in forecasting, but specializes in that task alone.

Therefore:

- For tasks that require general time-series understanding (e.g., imputation, anomaly detection), **encoder** models may be more suitable.
- However, for time-series forecasting, decoders currently hold the lead.

This is why the authors shifted from Timer's generalist design to Timer-XL's specialization in forecasting. Both models are decoders, but the decoder architecture benefits the forecasting task!

Long-Context Forecasting

The primary advantage of Transformer models is their ability to handle a large context length.

Modern LLMs such as Gemini support up to 1M tokens. They're not flawless at this scale, but generally reliable up to 100k tokens.

Time-series models are far behind — Transformer and DL forecasting models often struggle beyond 1K tokens. Recent foundation models such as MOIRAI support up to 4K.

There are 2 issues to examine here:

- The maximum supported context length.
- How the model handles the increased context length, in terms of performance.

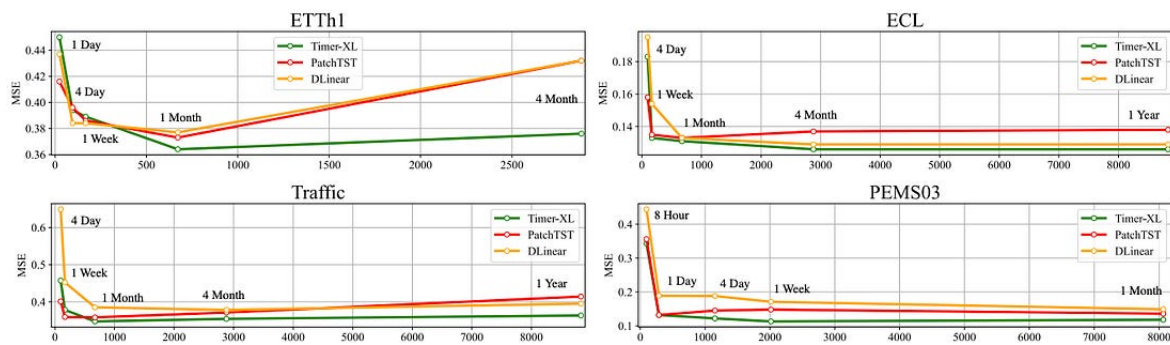


Figure 2: Performance of different models across context lengths. Prediction length: 96 datapoints (Source [1])

Figure 2 shows that Timer-XL handles increasing context better than other models.

For daily datasets such as Traffic, we can use up to a year's data (~8760 datapoints). This makes Timer-XL ideal for high-frequency forecasting — a configuration where foundation models often underperform, as discussed earlier.

Since we previously discussed Encoder-vs-Decoder models, let's mention how this paper explored how each architecture affects the efficiency of Long-Context Forecasting.

In Figure 3, they analyzed where the model focuses during prediction using attention maps:

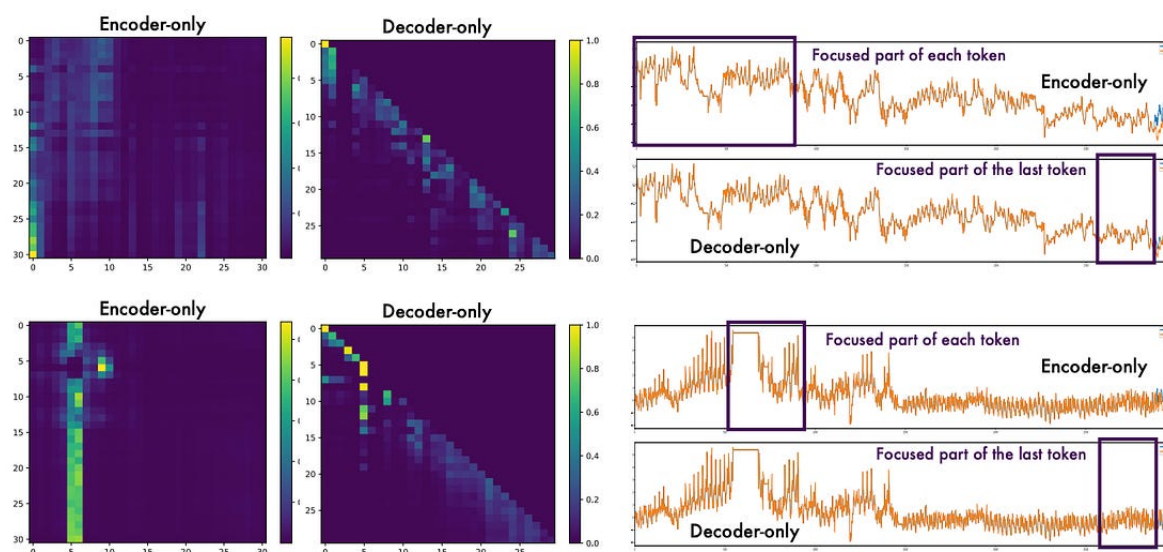


Figure 3: Interpretation of attention maps across a lookback window in encoder vs decoder architectures (Source [1])

Let's interpret the above plots:

Encoder: Exhibits broad and diffuse attention across most of the sequence. Each token attends to many other tokens—this suggests less focus. Occasionally, they focus on irrelevant parts, completely missing the most recent datapoints.

Decoder: Sparse, triangular structure. Attention is mostly local and causal (as expected), but selective peaks suggest it occasionally “reaches back” further when useful.

***Encoder** models scatter their attention across the sequence. **Decoder** models focus on **recent tokens** but adaptively zoom in on **useful earlier ones**.*

TimeAttention: The “Secret Sauce” of Timer-XL

The attention mechanism powers the Transformer (a breakthrough in NLP), but in time series, it's a double-edged sword. We discussed this extensively [here](#).

In short:

1. Transformer Time Series models are prone to overfitting.
2. We cannot use raw attention as in NLP because self-attention is permutation-invariant (the order of tokens doesn't matter, which it should when we have temporal information).

Let's first examine how deep learning models address time series, focusing on SOTA methods from each year.

Preliminaries

In DL time series, a single data point or a patch (a group of consecutive data points) is referred to as a **token**.

Imagine a dataset of 3 time series, where **y** is the target variable and **x**, **z** are observed covariates. For an input size **T**, the sequences are formatted as:

- $y_1, y_2, y_3 \dots y_T$
- $x_1, x_2, x_3 \dots x_T$
- $z_1, z_2, z_3 \dots z_T$

Early Transformer models like PatchTST applied attention across the **time dimension**, e.g., between $\langle y_1, x_1, z_1 \rangle, \langle y_2, x_2, z_2 \rangle \dots \langle y_N, x_N, z_N \rangle$.

This approach has a few disadvantages — e.g., misses lagged information

across the feature dimension.

Later models, such as iTransformer, addressed this by applying attention across the **feature/variante dimension**, between $\langle y_1, y_2, y_3, \dots y_N \rangle$, $\langle x_1, x_2, x_3, \dots x_N \rangle$, $\langle z_1, z_2, z_3, \dots z_N \rangle$. You can see the difference between the 2 approaches in **Figure 4** below:

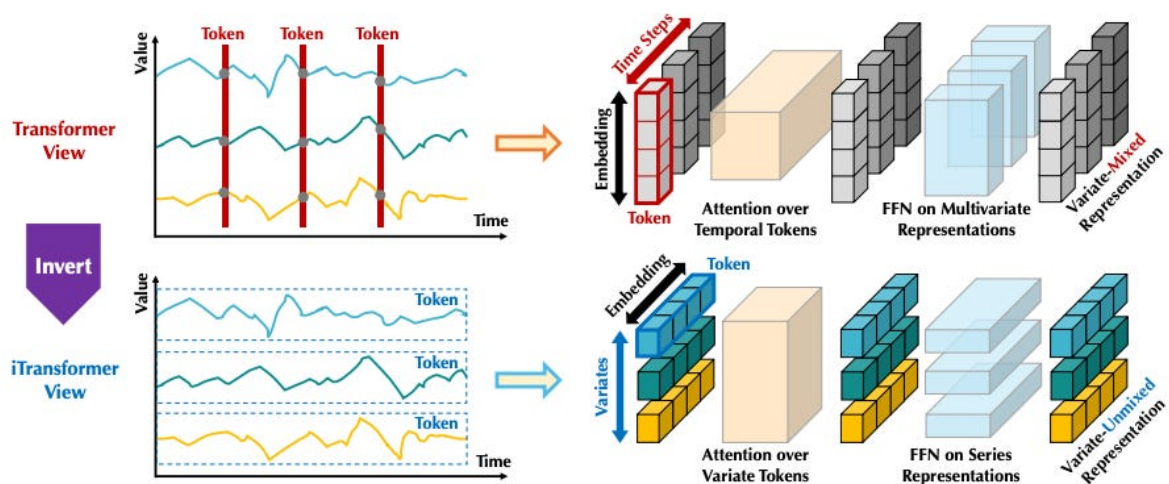


Figure 4: Attention across the time dimension(top) and feature/variante attention in iTransformer(bottom) ([Source](#))

As you might have guessed, we can further improve performance by combining both approaches. For instance, CARD (an encoder-only model) employs a sequential 2-stage attention mechanism for cross-time, cross-feature dependencies (**Figure 5**):

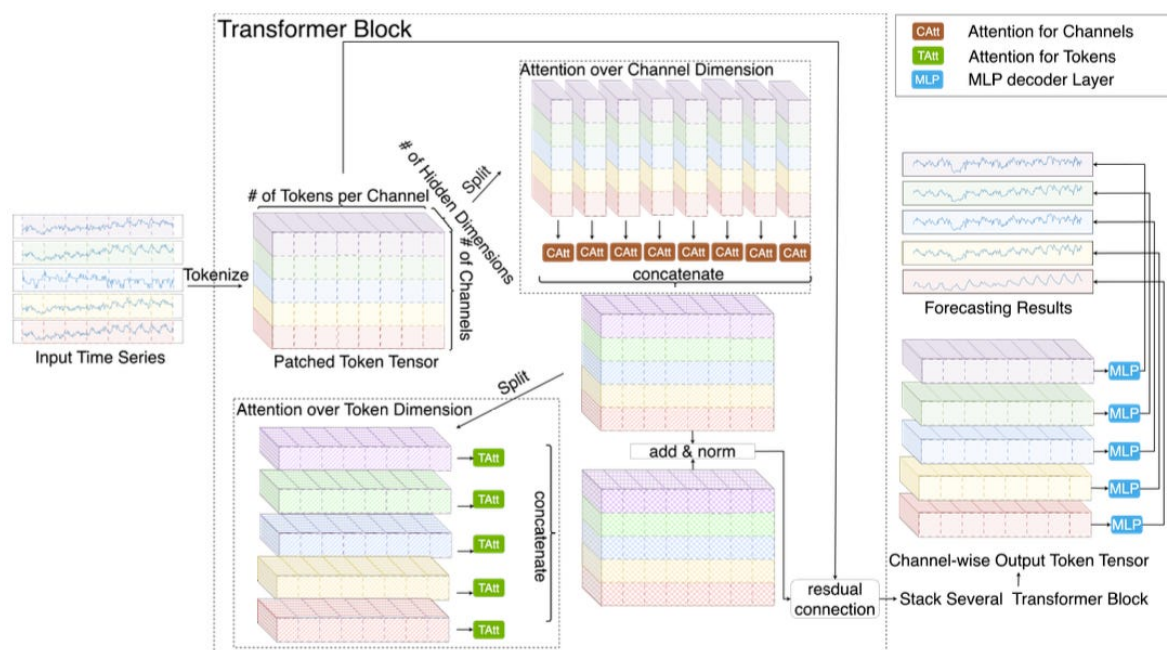


Figure 5: Top-level view of CARD's architecture, applying attention across both time/channel and token/feature dimension ([Source](#))

However, this method is slow — due to its sequential 2-stage design.

MOIRAI took things to the next level by introducing **Any-Variate Attention**, a novel attention mechanism adapted for pretrained models.

Any-Variate captures temporal relationships among datapoints while preserving permutation invariance across covariates. We discussed this mechanism in detail [here](#).

Next, we dive into Timer-XL's *TimeAttention*.

Enter TimeAttention

MOIRAI is a masked encoder model, so Any-Variate Attention can't be used as-is in decoder-only models like Timer-XL.

Decoder-only models use *causal self-attention* — attending only to previous tokens. In contrast, vanilla quadratic self-attention is bidirectional.

To address this, Timer-XL introduces a causal variant of MOIRAI's attention called **TimeAttention**.

Specifically, TimeAttention introduces:

- *Rotary positional embeddings (ROPE)* to capture time dependencies
- **Binary biases (ALIB)** to capture dependencies among variates.
- Causal self-attention.

The goal of TimeAttention is:

1. **No permutation invariance for temporal information** — the ordering of datapoints/temporal tokens should matter.
2. **Permutation invariance between the variates/features** — ordering of the variates should not matter (e.g., if we have 2 covariates x_1 and x_2 , it does not matter which is first, just the relationship between them). This achieves **permutation equivalence**.

The attention score between the $(\mathbf{m}, \mathbf{i})$ -th query and the $(\mathbf{n}, \mathbf{j})$ -th key, where $\mathbf{i}, \mathbf{j}$ represent the time indexes and $(\mathbf{m}, \mathbf{n})$ the variate indexes, is calculated as follows:

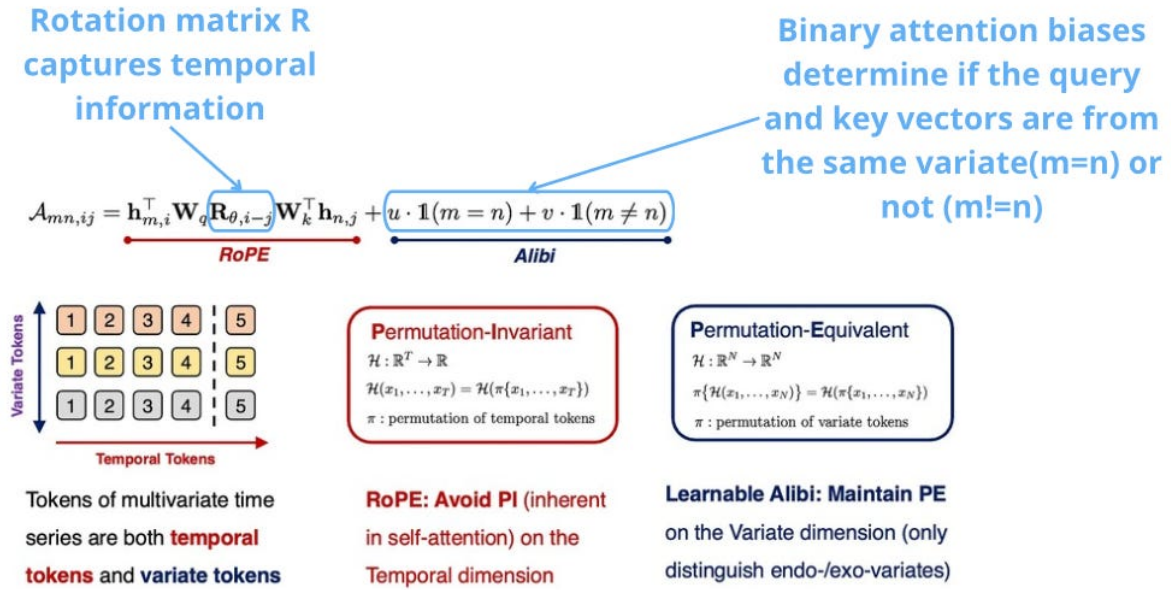


Figure 6: The TimeAttention equation of Timer-XL (Source [4], Annotated by author)

TimeAttention enables Timer-XL to handle univariate, multivariate, and context-informed forecasting.

Given N variables for a dataset, the model flattens the 2D input [observations, N] into 1D indices and calculates the attention-score by respecting permutation invariance in the time dimension and permutation equivariance in the variate dimension.

Standard self-attention is pairwise, resulting in quadratic cost. A key difference from MOIRAI is the use of **causal attention**, where each patch attends to previous points of its own signal, or to other time signals.

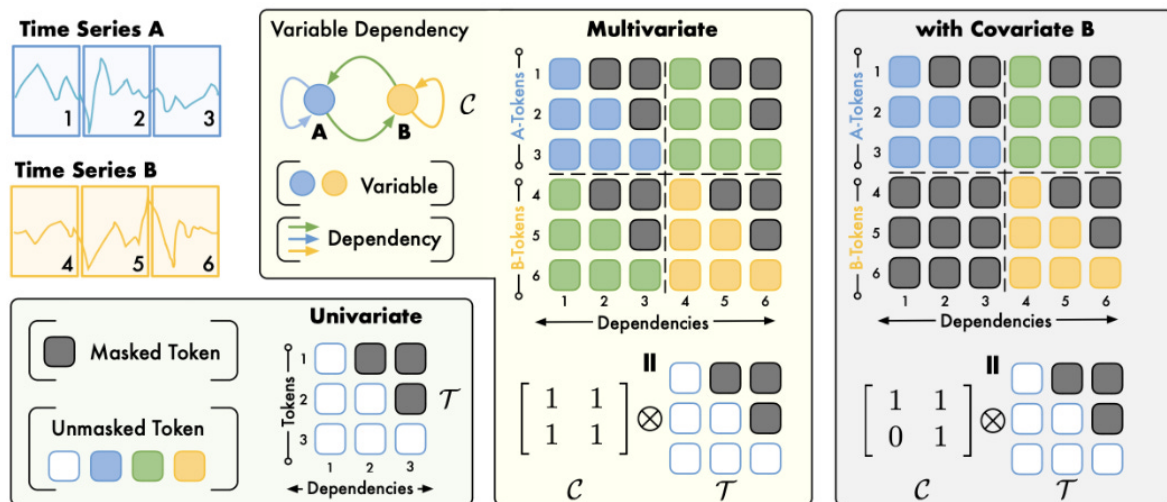


Figure 7: TimeAttention uses a temporal mask $\mathcal{T}$ to preserve causality in univariate series. For multivariate data, an all-in-one variable dependency matrix $\mathcal{C}$ is combined with $\mathcal{T}$ via the Kronecker product to form a block mask capturing cross-variable temporal dependencies. This approach also generalizes to univariate and covariate-informed settings with predefined dependencies. (Source [1])

In **Figure 7's** example, say we have 2 time series, A and B, each consisting of 3 tokens. The interactions are shown more clearly in **Figure 8:**

- Time Series A consists of tokens 1, 2, and 3
- Time Series B consists of tokens 4, 5, and 6

For Token 2 of Time Series A, the following attention scores are calculated:

- token2 $\leftrightarrow$ token2 (to itself)
- token2 $\leftrightarrow$ token1
- token2 $\leftrightarrow$ token4
- token2 $\leftrightarrow$ token5

The first 2 attention scores are within time series A, while the last 2 are

within time series B. The TimeAttention formula can distinguish different time series and preserve permutation-equivalence.

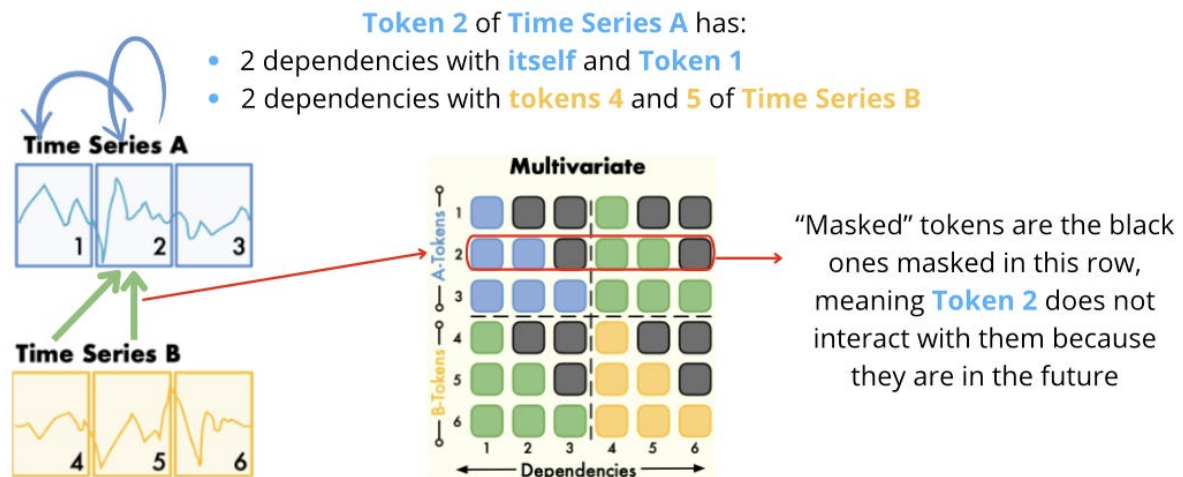


Figure 8: The 2nd red row corresponds to all interactions with token2 of Time-Series A with all the other tokens. There are 4 interactions in total (Source [1], annotated by author)

While TimeAttention may seem complex and somewhat arbitrary, ablation studies performed by the authors show it offers optimal performance in this setup.

Note: When we mention tokens, we don't mean individual datapoints. Timer-XL uses **patches** as tokens — so attention scores are computed between patches, not individual datapoints.

Enter Timer-XL

The full architecture of Timer-XL is shown below:

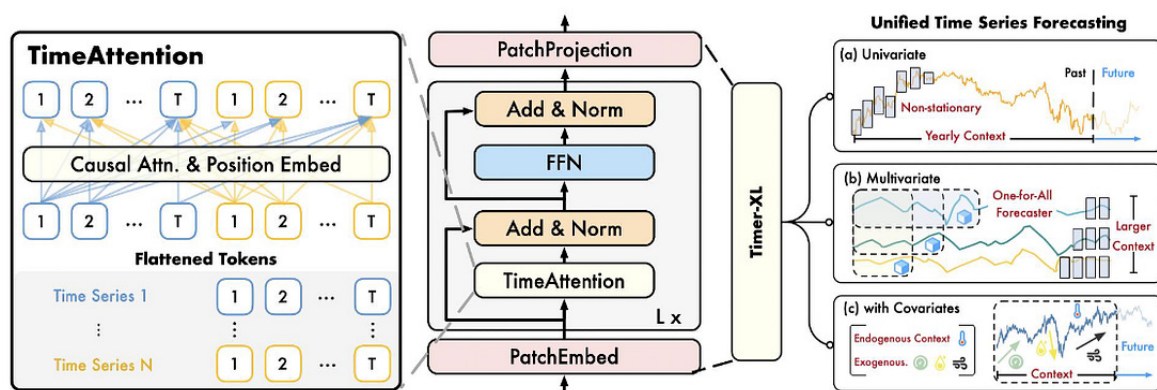


Figure 9: Top-level view of Timer-XL (Source [1])

Timer-XL focuses on long-context and long-term forecasting using TimeAttention. Moreover, Timer-XL uses the latest perks of advanced LLMs like FlashAttention:

FlashAttention speeds up Transformer attention and reduces memory use by optimizing matrix operations and eliminating redundant steps.

Interestingly, Timer-XL does **not** benefit from **Reversible Instance Normalization** (RevIN) — a technique often used to handle distribution shifts in time-series data. The authors attribute this to the window-centric structure of Timer-XL, which operates across different variates (likely due to the flattening process).

Let's now check some interesting benchmarks!

Multivariate Forecasting Benchmark

Timer-XL is primarily a zero-shot foundation model, but it can also be trained from scratch on specific datasets.

In **Table 1**, Timer-XL is benchmarked against popular DL models across well-known datasets. For each dataset, average MAE and MSE are reported using a **rolling-forecast** evaluation across the test set.

Rolling Forecasting: In this approach, each model is trained with 672 input steps to produce $N = (96, 192, 336, \text{ or } 720)$ output steps. The previous window slice of real observations is then recursively fed back as input until the target forecast length N is reached. **Table 1** shows the average score across all prediction lengths.

Models	Timer-XL (Ours)		Timer (2024c)		UniTST (2024a)		iTransformer (2023)		DLinear (2023)		PatchTST (2022)		TimesNet (2022)		Stationary (2022b)		Autoformer (2021)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ECL	0.155	0.246	<u>0.161</u>	<u>0.251</u>	0.163	0.257	0.164	0.258	0.165	0.265	0.169	0.268	0.201	0.303	0.265	0.358	0.289	0.379
ETTh1	0.409	0.430	0.418	0.436	0.429	0.447	0.421	0.445	0.426	0.444	<u>0.412</u>	<u>0.435</u>	0.495	0.491	0.505	0.513	0.517	0.528
Traffic	0.374	0.255	<u>0.384</u>	<u>0.259</u>	0.385	0.265	<u>0.384</u>	0.274	0.423	0.298	0.391	0.275	0.602	0.322	0.630	0.347	0.684	0.433
Weather	0.240	0.273	0.232	<u>0.270</u>	<u>0.231</u>	0.272	0.266	0.291	0.239	0.291	0.226	0.268	0.264	0.293	0.308	0.329	0.435	0.455
Solar-Energy	0.198	0.249	0.233	0.249	0.241	0.275	0.213	0.291	0.222	0.283	<u>0.202</u>	<u>0.269</u>	0.213	0.295	0.254	0.315	0.265	0.325

Table 1: The best model in each category is highlighted in bold red, and the 2nd best is in underlined blue (Source[1])

Key takeaways:

- Timer-XL achieves the best average scores.
- Previous SOTA models like PatchTST and iTransformer ranked among the top 2 in a few cases.
- Timer-XL and Timer — the only decoder-only models — perform the best.

The benchmark is further extended to include time-series models from other domains. Below are results from the **GTWSF dataset** (a wind speed forecasting challenge):

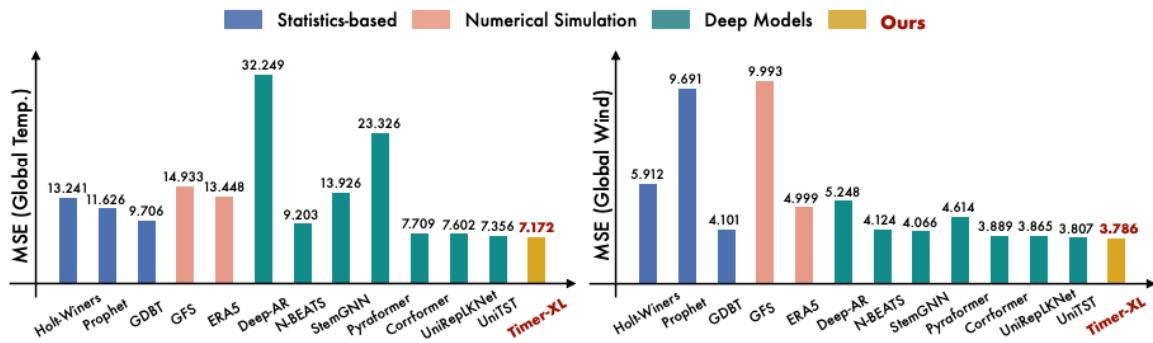


Figure 10: Comparing different types of models on the GTWSF dataset (Source[1])

We observe:

- Timer-XL achieves the top overall score.
- Most DL models perform well, though some fail spectacularly!
- GDBT is not a statistics-based method, probably refers to Boosted Trees - in fact, it would be nice if the popular GDBT models like LightGBM were present here.
- A **Seasonal Naive** baseline would've been a useful inclusion as well.

Covariate-Informed Time Series Forecasting

In multivariate forecasting, each time series is treated as a separate channel. The model captures their inter-dependencies and jointly forecasts all time series.

In covariate-informed forecasting, we predict a target series using others as covariates. These may be past-observed, future-known, or static. That's the setup discussed here.

An example of covariate-informed forecasting is shown in **Figure 7**

(right). In that case we predict variable A based on its dependency (green arrow) from B .

Table 2 shows results on the **EPF dataset** (electricity price forecasting task, Lago et al., 2021). The authors include both the current Timer-XL and an encoder Timer-XL variant (the one with the non-causal tag).

Models	Timer-XL (Ours)		Timer-XL (Noncausal)		TimeXer (2024b)		iTransformer (2023)		DLinear (2023)		PatchTST (2022)		Crossformer (2022)		TimesNet (2022)		Autoformer (2021)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
NP	0.234	0.262	<u>0.237</u>	<u>0.265</u>	0.238	0.268	0.265	0.300	0.309	0.321	0.267	0.284	0.245	0.289	0.250	0.289	0.402	0.398
PJM	<u>0.089</u>	0.187	0.092	<u>0.188</u>	0.088	0.188	0.097	0.197	0.108	0.215	0.106	0.209	0.149	0.198	0.097	0.195	0.168	0.267
BE	0.371	0.243	0.410	0.279	<u>0.379</u>	0.243	0.394	0.270	0.463	0.313	0.403	<u>0.264</u>	0.436	0.294	0.419	0.288	0.500	0.333
FR	0.381	0.204	0.406	0.220	<u>0.384</u>	<u>0.208</u>	0.439	0.233	0.429	0.260	0.411	0.220	0.440	0.216	0.431	0.234	0.519	0.295
DE	0.434	0.415	<u>0.435</u>	0.415	0.440	<u>0.418</u>	0.479	0.443	0.520	0.463	0.461	0.432	0.540	0.423	0.502	0.446	0.674	0.544
Average	0.302	0.262	0.316	0.273	<u>0.306</u>	<u>0.265</u>	0.335	0.289	0.366	0.314	0.330	0.282	0.362	0.284	0.340	0.290	0.453	0.368

Table 2: Covariate Forecasting Benchmark on the EPF Dataset (Source[1])

Observe that:

- Timer-XL achieves the best results.
- The decoder version (Timer-XL) outperforms the encoder-based Timer-XL.
- TimeXer is an encoder-based Transformer that uses cross-attention to model dependencies with exogenous variables, each treated as a separate token. This was a previous SOTA DL model — now outperformed by Timer-XL.

Zero-Shot Forecasting Benchmark

Finally, the authors also evaluate Timer-XL as a foundation model,

comparing it with other top time series models.

They perform univariate pretraining on the LOTSA dataset (used to pretrain MOIRAI) and UTSD. The latter was curated by the THUML research group and used to pretrain Timer. It contains [1B datapoints](#).

The same metrics and configurations as before are used (e.g., scores averaged across prediction_lengths = {96, 192, 336, 720}). This time, none of the models were trained on the below datasets (zero-shot forecasting).

Models	Timer-XL _{Base} (Ours)		Time-MoE _{Base} (2024)		Time-MoE _{Large} (2024)		Time-MoE _{Ultra} (2024)		Moirai _{Small} (2024)		Moirai _{Base} (2024)		Moirai _{Large} (2024)		TimesFM (2023)		MOMENT (2024)		Chronos _{Base} (2024)		Chronos _{Large} (2024)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	0.373	0.392	0.394	0.415	0.376	0.405	0.356	0.391	0.436	0.410	0.406	0.385	0.422	0.391	0.433	0.418	0.670	0.536	0.645	0.500	0.555	0.465
ETTm2	0.273	0.336	0.317	0.365	0.316	0.361	0.288	0.344	0.307	0.347	0.311	0.337	0.329	0.343	0.328	0.346	0.316	0.365	0.310	0.350	0.295	0.338
ETTh1	0.404	0.417	0.400	0.424	0.394	0.419	0.412	0.426	0.428	0.427	0.417	0.419	0.480	0.439	0.473	0.443	0.683	0.566	0.591	0.468	0.588	0.466
ETTh2	0.347	0.388	0.366	0.404	0.405	0.415	0.371	0.399	0.361	0.384	0.362	0.382	0.367	0.377	0.392	0.406	0.361	0.409	0.405	0.410	0.455	0.427
ECL	0.174	0.278	-	-	-	-	-	-	0.218	0.303	0.187	0.274	0.186	0.270	-	-	0.765	0.686	0.214	0.278	0.204	0.273
Weather	0.256	0.294	0.265	0.297	0.270	0.300	0.256	0.288	0.275	0.286	0.287	0.281	0.264	0.273	-	-	0.294	0.326	0.292	0.315	0.279	0.306
1 st Count	15	10	2	1	3	0	10	7	0	0	0	5	1	10	0	1	2	0	0	0	0	2

Table 3: Zero-shot Forecasting benchmark (Source[1])

We notice the following:

- Timer-XL-base (the zero-shot version) scores the most wins.
- Time-MOE-ultra ranks 2nd, followed by the MOIRAI models.
- Interestingly, MOIRAI-Base takes some wins from **MOIRAI-Large** — a pattern seen in other benchmarks. Despite having fewer parameters, MOIRAI-Base often matches or outperforms MOIRAI-Large.
- Unfortunately, the SOTA versions of existing pretrained models are not included. For example, newer versions like TimesFM-2.0, Chronos-Bolt, MOIRAI-MOE, and other popular models, such as Tiny-Time-Mixers (TTM) and TabPFN-TS, are missing.

The authors have included the complete benchmark of foundation models in **Table 13** of the paper. Feel free to read the paper for more details. In those results, I noticed that Timer-XL performs slightly better with longer prediction lengths compared to the other models.

Interpretability

Few time-series models explore interpretability, especially the DL models. Timer-XL authors demonstrate the benefits of attention by showing how it captures interdependencies among time series variables.

Figure 11 shows the average attention score across all attention matrices. The matrix is 10×10, which corresponds to the attention score pairs among 10 variables. The 2 tiles in the figure show the relationship between attention and correlation. The pair of variables <6,7> has a higher attention score than <6,2>—which correctly translates to the pair <6,7> having a higher correlation.

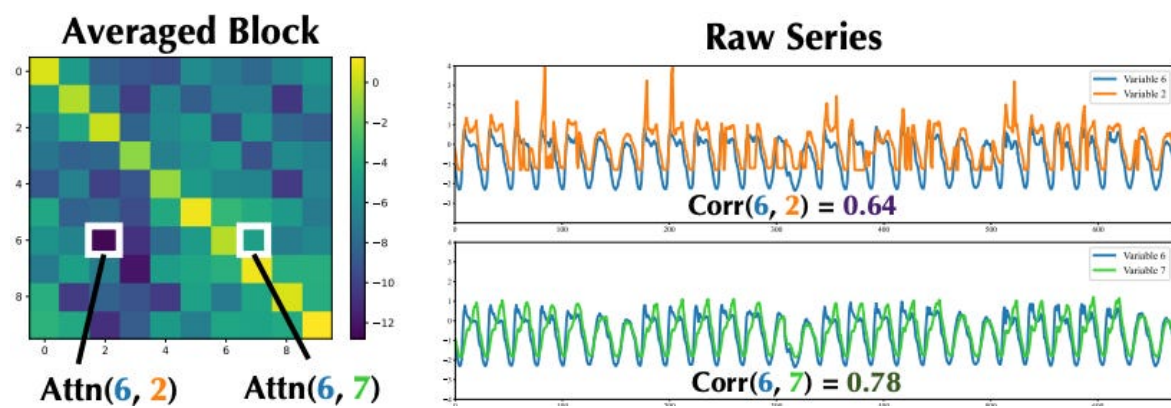


Figure 11: Relationship between attention and correlation, showing 2 pairs of variables (Source [1])

Hence, we conclude:

The Transformer’s attention mechanism learns to interpret the inner dependencies between the different variables.

This isn’t a new pattern—we have seen many Transformer forecasting models using this trick. In this case, it means Timer-XL’s attention mechanism works as expected and correctly captures the variate dependencies.

Univariate Forecasting Tutorial

At the time of writing this article, the authors have released the univariate pretrained version of Timer-XL.

Thus, I used this version for now to evaluate Timer-XL on the ETTh2 dataset and the latest month of the SP&500 index.

The results are quite promising. **Figure 12** shows Timer-XL’s predictions on the ETTh2 dataset:

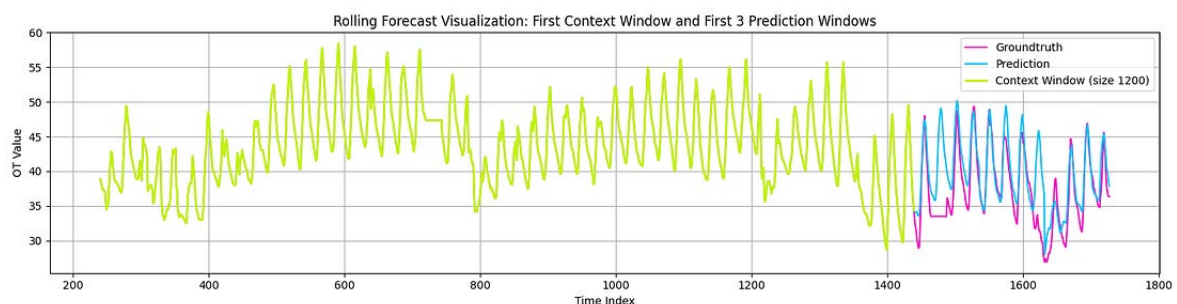


Figure 12: Rolling Forecast Evaluation on ETTh2 using Timer-XL-Base (Image by author)

I don’t want to make this article very long, so we’ll go through the tutorials

on Part 2. You can also find them in the [AI Projects Folder](#) (Project 18 and Project 19). Stay tuned!

Thank you for reading!

Horizon AI Forecast is a reader-supported newsletter, featuring occasional bonus posts for supporters. Your support through a paid subscription would be greatly appreciated.

Upgrade to paid

References

- [1] Liu et al. [Timer-XL: Long-context Transformers For Unified Time Series Forecasting](#) (March 2025)
- [2] Liu et al. Timer: [Generative Pre-trained Transformers Are Large Time Series Models](#) (October 2024)
- [3] [THUML @ Tsinghua University](#)
- [4] Liu et al. [Timer-XL ICRL2025 Presentation Slides](#)

Thank you for reading [AI Horizon Forecast](#) by [Nikos Kafritsas](#)! If you liked this article, feel free to share it!

Share this article!



LIKE



COMMENT



RESTACK

© 2025 Nikos Kafritsas

[Unsubscribe](#)

Get the app



Start writing